

Ingeniería de Software I

Avance de proyecto # 2 – Requisitos y planeación inicial

Fecha de asignación: viernes 28 de agosto de 2026.

Docente: Juan Pablo Bustamante Moreno.

Escuela Tecnológica Instituto Técnico Central (ETITC).

En la primera entrega, cada equipo definió una idea de producto, identificó el problema, los usuarios, la propuesta de valor, el alcance preliminar y los acuerdos de trabajo. Ahora deberán convertir esa definición inicial en una primera especificación de requisitos y en un backlog de producto organizado.

El objetivo es entender qué debe hacer la solución, para quién, bajo qué condiciones de calidad y cómo se dividirá el trabajo antes de iniciar las fases de prototipado, modelo de datos, arquitectura y desarrollo por Sprints.

1. Requisitos funcionales

Identifiquen las funcionalidades principales que debe ofrecer el sistema para responder al problema definido. Tras ello, redacten **entre ocho y quince** requisitos funcionales, según el alcance de su propuesta.

Recuerden que un requisito funcional expresa una capacidad que el sistema debe permitir realizar a un usuario, actor o sistema externo.

Código	Requisito funcional	Actor o usuario relacionado
RF-01		
RF-02		
RF-03		
...		

Recomendaciones:

- Eviten mezclar varias funcionalidades grandes en un solo requisito. Un ejemplo adecuado de un requisito sería “RF-01. El usuario podrá consultar el estado de sus solicitudes registradas”, y no “RF-01. El usuario podrá gestionar solicitudes”.
- No incluyan aquí tareas técnicas como “integrar API”, “configurar base de datos” o “desarrollar un módulo”. Cada requisito debe describir una acción, capacidad o comportamiento observable.
- Si una funcionalidad depende de permisos, roles o reglas, incluyan la funcionalidad aquí y documenten la condición específica en las reglas de negocio (esto aparece más adelante).

2. Requisitos no funcionales y escenarios de calidad

Los requisitos no funcionales describen condiciones de calidad que el sistema debe cumplir. No indican solo qué hace el software, sino cómo debe comportarse ante determinadas situaciones.

Seleccionen **entre siete y doce** requisitos no funcionales relevantes para el proyecto. Estos deben ser claros y verificables; es decir, indicar que “el sistema debe ser seguro” no funciona como requisito, porque resulta amplio y ambiguo.

Código	Requisito no funcional	Atributo de calidad
RNF-01		
RNF-02		
RNF-03		
...		

Luego, a partir de esta definición, seleccionen **al menos tres** requisitos no funcionales prioritarios (de categorías distintas) y conviértanlos en escenarios de calidad verificables. Elijan los que más impactan a sus usuarios y/o su propuesta de valor.

Usen la siguiente estructura:

Elemento	Descripción
Requisito no funcional asociado	RNF-XX
Atributo de calidad	
Fuente del estímulo	Usuario, administrador, sistema externo, proceso automático, atacante, etc.
Estímulo	Acción, evento o condición que ocurre
Artefacto afectado	Funcionalidad, módulo, interfaz, API, base de datos, etc.
Entorno	Condición en la que se presenta el estímulo
Respuesta esperada	Cómo debe responder el sistema
Medida de respuesta	Cómo se comprobará el cumplimiento

3. Restricciones y reglas de negocio

En esta sección se documentan los límites y las reglas que el sistema debe respetar.

Por un lado, las restricciones son condiciones que limitan el desarrollo o las opciones de solución. Pueden estar relacionadas con tiempo, tecnología, datos, infraestructura, aspectos legales o recursos disponibles. Por ejemplo: “El sistema debe utilizar autenticación institucional”.

Incluyan **entre cuatro y siete restricciones** que tendrá su sistema (procuren ser específicos e ir más allá de las condiciones temporales de la asignatura).

Código	Restricción	Tipo
RST-01		Técnica / tiempo / datos / infraestructura / legal / organizacional
RST-02		

RST-03		
...		

Por otra parte, las reglas de negocio son normas del problema o del contexto que el sistema debe respetar: describen normas, políticas o condiciones de operación. Por ejemplo: “Una reserva solo puede cancelarse 24 horas antes de la hora programada”.

Incluyan entre **cuatro y siete reglas de negocio** que debe respetar su proyecto.

Código	Regla de negocio	Relación con requisito funcional
RN-01		RF-XX
RN-02		RF-XX
RN-03		RF-XX
...		

4. Historias de usuario y Product Backlog

A partir de los requisitos funcionales, construyan el Product Backlog inicial del proyecto. Este debe permitir visualizar qué funcionalidades se desarrollarán primero, cuáles requieren mayor análisis y cuáles pueden quedar fuera de la primera versión.

Para construir el backlog, primero tendrán que **definir las historias de usuario** derivadas de sus requisitos funcionales.

Usen esta estructura:

Como [rol], quiero [funcionalidad], para [beneficio].

Cada historia debe incluir **al menos dos criterios de aceptación** verificables. Redacten entre ocho y quince historias, según la complejidad del proyecto (debería haber una por cada requisito funcional).

Debido a la cantidad de texto, pueden usar la siguiente tabla como guía, pero no necesariamente deben aplicar dicho formato. De hecho, es recomendable que cada historia de usuario esté en cuadro o tabla separado, para facilitar su lectura.

ID	Épica asociada	Historia de usuario	Criterios de aceptación	Tamaño	Prioridad MoSCoW
HU-01	EP-01	Como..., quiero..., para...	1. Dado que... cuando... entonces... 2. Dado que... cuando... entonces...		Must / Should / Could / Won't
HU-02					
HU-03					
...					

Recomendaciones:

- No redacten historias desde el punto de vista del sistema; redacten desde el valor para un rol.
- Primero definan todas las historias de usuario y luego, aparte, hagan una lista con las épicas de su proyecto, según las asociaciones que vayan surgiendo.
- Estimen el tamaño relativo de cada historia usando Planning Poker o una escala equivalente. Si una historia tiene tamaño 13 o superior, revisen si debe dividirse en historias más pequeñas, convertirse en una épica o requerir un spike antes de estimarse.

- Prioricen cada elemento del backlog según aspectos como valor para usuarios, riesgos, dependencias, complejidad, urgencia y viabilidad durante el semestre.
- Incluyan criterios de aceptación para un comportamiento normal y, cuando corresponda, también criterios de error, validación, permisos o ausencia de datos. Usen el siguiente formato:

*Dado que [contexto],
cuando [acción o evento],
entonces [resultado esperado].*

Finalmente, construyan el tablero inicial en una herramienta de gestión de trabajo. Puede ser Trello, GitHub Projects, Jira, Notion u otra herramienta acordada por el equipo. Debe incluir como mínimo las columnas:

Product Backlog | To Do | In Progress | Review | Done

También, traten de agregar responsables, etiquetas, checklist con tareas técnicas y demás elementos. Esto con el fin de llevar una trazabilidad adecuada.

Incluyan en el documento:

- Captura de pantalla y enlace al tablero.
- Explicación breve del backlog.
- Identificación de al menos una dependencia o spike, si aplica.

5. Diagrama de casos de uso

Elaboren un diagrama de casos de uso inicial del sistema.

Elaboren una primera versión del diagrama de casos de uso del sistema. Este diagrama debe servir para delimitar el sistema y validar que los actores identificados se relacionan con las funcionalidades propuestas.

Incluyan:

- Frontera clara del sistema.
- Actores principales y secundarios.
- Casos de uso relacionados con funcionalidades relevantes.
- Relaciones **<<include>>** y **<<extend>>** solo cuando sean necesarias.
- Correspondencia razonable con requisitos funcionales e historias de usuario.

No se busca un diagrama excesivamente detallado. El objetivo es que sea legible y útil para validar el alcance del sistema.

Trazabilidad esperada:

Requisito funcional	Historia de usuario	Caso de uso
RF-01	HU-01	CU-01
RF-02	HU-02	CU-02
RF-03	HU-03	CU-03
...		

6. Mapa de stakeholders

Tras la definición de usuarios que se hizo en el primer avance, y complementando con los actores que aparecieron durante el análisis de requisitos, construyan una representación gráfica de los stakeholders del proyecto.

Cada equipo puede seleccionar el formato que considere más adecuado. Algunas opciones son:

- Matriz de interés vs. poder.
- Diagrama de Venn.
- Mapa de actores (diagrama en círculos concéntricos).
- Red de relaciones.
- Cualquier otra representación gráfica justificada.

El mapa de actores debe incluir, según aplique:

- Usuarios principales.
- Usuarios secundarios.
- Administradores.
- Cliente, patrocinador o persona que valida la propuesta.
- Personas u organizaciones afectadas.
- Sistemas, APIs o servicios externos.
- Entidades que definen restricciones o reglas.

En el gráfico, se debe poder ver con claridad el nombre o tipo de stakeholder, su nivel de interés, su nivel de poder, influencia o capacidad de decisión y su relación principal con el sistema.

Después del gráfico, incluyan un párrafo breve que responda:

¿Quién(es) tiene(n) mayor interés en la solución, quién(es) tiene(n) mayor poder para modificar requisitos o prioridades, y qué actor(es) externo(s) puede(n) condicionar el proyecto?

Entrega

Actualicen el documento compartido de OneDrive creado para el Avance de Proyecto #1. No deben crear un archivo nuevo: el documento del proyecto va a crecer de manera progresiva durante todo el semestre.

Este avance no se califica de forma independiente. Será revisado y retroalimentado durante el Sprint 0, y hará parte de la evidencia evaluada en la sustentación del primer corte. Se sugiere completar lo que más puedan de aquí al próximo viernes.